

Topic: Types of Queue - Circular Queue, Priority Queue

Q. Explain the limitations of a simple linear queue. How do a Circular Queue and a Priority Queue overcome specific challenges? Describe their working principles.

****Definition to Remember****

A ****Circular Queue**** is a data structure that follows the FIFO principle but is implemented using a circular array. It overcomes the limitation of a linear queue where spaces are wasted at the front. It is critical for applications where memory efficiency is important.

Queue Types: Circular & Priority Queues

In a standard linear array queue, once the `rear` pointer reaches the maximum index ($\text{SIZE} - 1$), no new elements can be enqueued. This happens even if spaces are empty at the front due to earlier dequeue operations, leading to severe memory wastage.

2. Circular Queue

Solution: The queue forms a logical circle. The next position after the last index is index 0.

* ****Working Principle:**** Uses the ****modulo operator**** ($\% \text{SIZE}$) to wrap pointers around.

* Next

[0] <-- front

/ \

3. Priority Queue

Problem: A standard queue strictly processes the oldest element first. Some real-world tasks (like CPU interrupts) are urgent and cannot wait in a FIFO line.

Solution

* ****Working Principle:****

* Element

****Must-Write Points (for 10 marks)****

1. ****Linear Queue****

*****Quick Recall*****

`Linear
served L

Complete Executable C Code: Circular Queue Array Implementation

```
#include <stdio.h>
```

```
#define MAX 5
```

```
struct CircularQueue {
```

int item

```
void initQueue(struct CircularQueue *q) {
```

q->front

```
int isFull(struct CircularQueue *q) {
```

return (

```
int isEmpty(struct CircularQueue *q) {
```

return 0

```
void enqueue(struct CircularQueue *q, int val) {
```

if (isFull

```
int dequeue(struct CircularQueue *q) {
```

```
void display(struct CircularQueue *q) {
```

```
int main() {
```

```
    enqueue(&q, 10);
```

```
    printf("Dequeued: %d\n", dequeue(&q));
```

```
    enqueue(&q, 50);
```

```
    return 0;
```

```
if (isEm
```

```
if (isEm
```

```
struct C
```

```
enqueu
```

```
printf("D
```

```
enqueu
```

```
}
```